

New Clarin Sports Arena POS - Application Guide

This document describes the architecture, features, and user flows of the New Clarin Sports Arena web-based POS application.

1. Overview

The app is a full-stack POS system built with an Express backend and a React + Vite frontend. It supports staff and vendor authentication, product inventory management, point-of-sale checkout, dashboard metrics, and exportable reports.

2. Architecture

Backend:

- Node.js + Express server under server/ - SQLite data persistence using better-sqlite3 in production - In-memory test database shim in server/db.js for fast test execution

Frontend:

- React application under client/src - Vite-based development server - Main views: Dashboard, POS, Reports

Database and data model:

- Users, roles, vendors, products, deliveries, delivery items, inventory movements, sales, and sale items - Core entities are created and seeded for both test and runtime environments

3. Authentication and Authorization

Users log in through /api/auth/login using email and password. The backend issues a JWT token valid for 8 hours.

Roles and access:

- ADMIN: full access to products, inventory, reports, dashboard, deliveries, and sales. - STAFF: full POS and product management access except vendor-specific restrictions. - VENDOR: read-only access to deliveries, vendor dashboard, and vendor reports. - Authorization is enforced in middleware/auth.js using requireRole() and authMiddleware().

4. Key Backend API Routes

Route	Description
POST /api/auth/login	Authenticate staff or vendor and receive JWT token.
GET /api/products	List products with optional search query and stock status.
POST /api/products	Create new product with optional initial stock and inventory movement.
PUT /api/products/:id	Update product metadata and pricing.
PATCH /api/products/:id/activate	Enable or disable a product.
POST /api/products/:id/stock-movements	Apply STOCK_IN, STOCK_OUT, or ADJUSTMENT inventory movements.
POST /api/sales	Create a POS sale transaction, decrement stock, record sale items, and insert inventory movement.
GET /api/dashboard	Load either operational or vendor dashboard metrics.

GET /api/reports/sales	Fetch sales report data.
GET /api/reports/sales/csv	Export sales report CSV.
GET /api/reports/inventory	Fetch inventory report data.
GET /api/reports/inventory/csv	Export inventory report CSV.
GET /api/reports/vendor-deliveries	Fetch vendor deliveries report.
GET /api/deliveries	List deliveries with filtering by date and vendor.
GET /api/deliveries/:id	Get delivery details and items.
GET /api/users/me	Load current authenticated user profile.

5. POS Flow

The POS checkout page allows staff to search products, add them to a cart, set quantities, and complete sales.

Key POS flow steps:

1. Load products via GET /api/products. 2. Add items from the product list to the cart. 3. Adjust item quantity within available stock. 4. Submit the cart to POST /api/sales. 5. On success, receive sale_id and total_amount, clear cart, and display transaction confirmation.

6. Sales Transaction Logic

On the backend, sales are processed in a transactional wrapper that performs:

- Validation of cart items and stock availability - Computation of total sale amount - INSERT into sales table - INSERT sale_items rows for each product - UPDATE products current_stock by subtracting sale quantity - INSERT inventory_movements with movement_type = SALE / STOCK_OUT - Return sale_id and total_amount to frontend

7. Inventory and Stock Management

Products expose current_stock and minimum_stock. The frontend renders stock status, and backend APIs enforce inventory movements.

Product status categories:

- OK: current_stock > minimum_stock - LOW_STOCK: current_stock > 0 && current_stock <= minimum_stock - OUT_OF_STOCK: current_stock <= 0

8. Dashboard Features

Dashboard shows operational metrics for ADMIN/STAFF users and vendor-specific summaries for VENDOR users.

Operational metrics include:

- Today's total sales - Transaction count - Items sold today - Total current inventory - Low-stock and out-of-stock counts - Recent sales list - Recent deliveries list - Low-stock product table

Vendor dashboard includes:

- Today's delivery count for the vendor - Today's delivery total amount - Recent deliveries for the vendor

9. Reports

The Reports page supports three report types and CSV export.

Report types:

- Sales report: transaction-level sales data with staff, items, quantity, and total. - Inventory report: stock-in, stock-out, current stock, minimum stock, and status. - Vendor deliveries: filtered delivery summaries for the selected vendor or administrator view.

CSV export is available for sales and inventory reports via `/api/reports/sales/csv` and `/api/reports/inventory/csv`.

10. Setup and Run Commands

Backend:

```
cd server && npm install npm run dev
```

Frontend:

```
cd client && npm install npm run dev
```

The backend listens by default on port 4000 and exposes `/api` routes. The frontend uses Vite and proxies API requests to `/api`.

11. Test and Seed Data

The server includes an in-memory test DB for Node tests. In test mode, the database is seeded with roles, users, vendors, products, deliveries, and inventory entries.

Login credentials for seeded users:

- ADMIN: `admin@clarin.local` / `Admin123!` - STAFF: `staff@clarin.local` / `Staff123!` - VENDOR (Clarin Beverages): `vendor@clarin.local` / `Vendor123!` - VENDOR (Arena Drinks Supplier): `vendor2@clarin.local` / `Vendor123!`

12. File Structure

Key files and folders:

- `server/app.js`: Express app setup and route mounting - `server/index.js`: server startup script - `server/db.js`: database initialization, schema, and in-memory DB shim - `server/routes/auth.js`: login route - `server/routes/products.js`: product CRUD and stock movements - `server/routes/sales.js`: sale transaction processing - `server/routes/reports.js`: report data and CSV export - `server/routes/dashboard.js`: dashboard metrics - `client/src/App.jsx`: login and view routing - `client/src/POS.jsx`: POS checkout UI - `client/src/Dashboard.jsx`: dashboard UI - `client/src/Reports.jsx`: report UI

13. Important Notes

- All authenticated requests require the Authorization header: `Bearer .` - Sale creation uses atomic transactions when supported to keep stock updates and sale records consistent. - Inventory movements are recorded for stock changes from sales and manual stock adjustments.

14. How to Use the App

This section explains the primary user experience for staff, admins, and vendors when using the web application.

Staff / Admin workflow:

1. Open the web client and log in with staff or admin credentials. 2. Use the Dashboard tab to review sales, inventory, and recent activity. 3. Navigate to the POS tab to search products, add items to the cart, adjust quantities, and complete a sale. 4. Use the Reports tab to select either Sales, Inventory, or Vendor Deliveries reports, filter by date range, and export CSVs as needed. 5. Use the Products API or future management views to create or update products, adjust stock, and monitor low/out-of-stock notifications.

Vendor workflow:

1. Log in with vendor credentials. 2. View the vendor dashboard summary of today's deliveries and recent delivery activity. 3. Use the vendor deliveries report to see delivery summaries for your vendor account. 4. Vendors can review delivery history without changing product inventory or sales records.

Appendix: Important Backend Flows

Login flow:

1. User submits email and password to POST /api/auth/login. 2. Backend verifies credentials against users table and bcrypt when available. 3. JWT token is returned on success.

Sale flow:

1. Frontend builds cart items and POSTS to /api/sales. 2. Backend validates product availability. 3. Backend writes sales, sale_items, updates product stock, and records inventory movements. 4. Backend returns transaction id and total amount.

Product stock movement flow:

1. POST /api/products/:id/stock-movements with type STOCK_IN, STOCK_OUT, or ADJUSTMENT. 2. Backend validates movement type and quantity. 3. Product current_stock is updated, and inventory_movements row is created.